

GA1

Nombre: Sara Lucía Tipáz Martinez

Clave: 25

Programa 1: Realizar un programa que inicialice una lista con 10 valores aleatorios (del 1 al 10) y posteriormente muestre en pantalla cada elemento de la lista junto con su cuadrado y su cubo.

```
E1_SLTM.py > ...
1  # Ejercicios de Listas
2  # E1_SLTM
3  # SLTM
4
5  import pyfiglet
6  from colorama import init, Fore, Back, Style
7
8  init()
9
10 titulo = pyfiglet.figlet_format("Sara")
11 print(Fore.CYAN + titulo + Style.BRIGHT)
12
13 print(Fore.MAGENTA + "Texto en color magenta")
14
15 import random
16 lista_numeros= []
17
18 # Primer recorrido para leer la lista
19 for indice in range(1,11):
20     lista_numeros.append(random.randint(1,10))
21 # Segundo recorrido para mostrar el resultado
22 for numero in lista_numeros:
23     print(numero, " ", numero **2, " ", numero **3)
```

Resultado:

```
PS D:\Python_EBGA1\Python_EB>Listas_SLTM> & C:/Users/HP/AppData/Local/Programs/Python/Python314/python.exe d:/Python_EBGA1/Python_EB/Listas_SLTM/E1_SLTM.py
3  9  27
10 100 1000
8  64  512
5  25  125
5  25  125
5  25  125
7  49  343
1  1  1
8  64  512
2  4  8
PS D:\Python_EBGA1\Python_EB>Listas_SLTM> 
```

Programa 2: Crea una lista e inicializala con 5 cadenas de caracteres leídas por teclado. Copia los elementos de la lista en otra lista pero en orden inverso, y muestra sus elementos por la pantalla. Observación: Al ejecutar su programa tiene que realizarlo con su nombre Completo en cada cadena y la cadena 5 es su clave.

```
E2_SLTM.py > ...
1  # Ejercicios de Listas
2  # E2_SLTM
3  # SLTM
4
5  import pyfiglet
6  from colorama import init, Fore, Back, Style
7
8  init()
9
10 titulo = pyfiglet.figlet_format("Sara")
11 print(Fore.CYAN + titulo + Style.BRIGHT)
12
13 print(Fore.MAGENTA + "Texto en color magenta")
14
15 # Crear listas vacias para llenarlas con 5 cadenas de caracteres
16 lista1 = []
17 lista2 = []
18
19 # Recorrido de Lista1 y leer cada elemento por teclado
20 for indice in range(1,6):
21     lista1.append(input("Ingrese la cadena %d:" % indice))
22
23 # Se copia la lista 1 en la lista2 en orden inverso
24 lista2 = lista1[::-1]
25
26 # Recorrido de Lista2 para mostrarlo
27 for cadena in lista2:
28     print(cadena)
```

Resultado:

```
PS D:\Python_EBGA1\Python_EB>Listas_SLTM> & C:/Users/HP/AppData/Local/Programs/Python/Python314/python.exe d:/Python_EBGA1/Python_EB/Listas_SLTM/E2_SLTM.py
Ingrese la cadena 1:Sara Lucia Tipaz Martinez
Ingrese la cadena 2:Sara Lucia Tipaz Martinez
Ingrese la cadena 3:Sara Lucia Tipaz Martinez
Ingrese la cadena 4:Sara Lucia Tipaz Martinez
Ingrese la cadena 5:25
25
Sara Lucia Tipaz Martinez
Sara Lucia Tipaz Martinez
Sara Lucia Tipaz Martinez
Sara Lucia Tipaz Martinez
PS D:\Python_EBGA1\Python_EB>Listas_SLTM>
```

Programa 3: Se quiere realizar un programa que lea por teclado las 5 notas obtenidas por un alumno (comprendidas entre 0 y 100). A continuación, debe mostrar todas las notas, la nota media, la nota más alta que ha sacado y la menor. Observación: Al ejecutar su programa la primera nota tiene que ser su clave.

```
E3_SLTM.py > ...
1  # Ejercicios de Listas
2  # E3_SLTM
3  # SLTM
4
5  import pyfiglet
6  from colorama import init, Fore, Back, Style
7
8  init()
9
10 titulo = pyfiglet.figlet_format("Sara")
11 print(Fore.CYAN + titulo + Style.BRIGHT)
12
13 print(Fore.MAGENTA + "Texto en color magenta")
14
15 # Lista vacia para agregar las notas
16 notas = []
17
18 # Recorrer lista para ingresar las notas
19 for indice in range(1,6):
20     while True:
21         nota = int(input("📝 Ingrese la nota %d: " % indice))
22         if nota >= 0 and nota <= 100 : break
23     notas.append(nota)
24
25 # Se muestran los resultados
26 print("Notas 📅: ", end="")
27 for nota in notas:
28     print(nota, " ", end="")
29 #Se imprimen las notas media, más alta y la menor
30 print("Nota media 📊: ", sum(notas)/len(notas))
31 print("Nota max 😄: ", max(notas))
32 print("Nota min 😞: ", min(notas))
```

Resultado:

```
PS D:\Python_EBGA1\Python_EB>Listas_SLTM> & C:/Users/HP/AppData/Local/Programs/Python/Python314/python.exe d:/Python_EBGA1/Python_EB/Listas_SLTM/E3_SLTM.py
Ingrese la nota 1: 25
Ingrese la nota 2: 34
Ingrese la nota 3: 98
Ingrese la nota 4: 78
Ingrese la nota 5: 95
Notas: 25 34 98 78 95 Nota media: 66.0
Nota max: 98
Nota min: 25
PS D:\Python_EBGA1\Python_EB>Listas_SLTM>
```

Programa 4: Programa que declare una lista y la vaya llenando de números hasta que introduzcamos un número negativo. Entonces se debe imprimir el vector (sólo los elementos introducidos). Observación: Al ejecutar su programa el primer número tiene que ser su clave.

```
E4_SLTM.py > ...
1  # Ejercicios de Listas
2  # E4_SLTM
3  # SLTM
4
5  import pyfiglet
6  from colorama import init, Fore, Back, Style
7
8  init()
9
10 titulo = pyfiglet.figlet_format("Sara")
11 print(Fore.CYAN + titulo + Style.BRIGHT)
12
13 print(Fore.MAGENTA + "Texto en color magenta")
14
15 # Crear lista vacia que se iran agregando valores positivos
16 lista = []
17
18 # Solicitar los numeros
19 numero = int(input("Ingrese un numero a la lista 📄: "))
20 # Comienza a verificar que sean solo positivos
21 while numero >= 0:
22     # Agregar elementos
23     lista.append(numero)
24     numero = int(input("Ingrese un numero a la lista ✅: "))
25
26 # Mostrar el listado de numeros ingresados
27 for numero in lista:
28     print(numero, " ", end="")
```

Resultado:

```
PS D:\Python_EBGA1\Python_EB>Listas_SLTM> & C:/Users/HP/AppData/Local/Programs/Python/Python314/python.exe d:/Python_EBGA1/Python_EB/Listas_SLTM/E4_SLTM.py
Ingrese un numero a la lista 📄: 25
Ingrese un numero a la lista ✅: 25
Ingrese un numero a la lista ✅: 46
Ingrese un numero a la lista ✅: -1
25 25 46
PS D:\Python_EBGA1\Python_EB>Listas_SLTM> |
```

Programa 5: Hacer un programa que inicialice una lista de números con valores aleatorios (10 valores), y posterior ordene los elementos de menor a mayor.

```
E5_SLTM.py > ...
1  # Ejercicios de Listas
2  # E5_SLTM
3  # SLTM
4
5  import pyfiglet
6  from colorama import init, Fore, Back, Style
7
8  init()
9
10 titulo = pyfiglet.figlet_format("Sara")
11 print(Fore.CYAN + titulo + Style.BRIGHT)
12
13 print(Fore.MAGENTA + "Texto en color magenta")
14
15 import random
16 # Creacion de una lista vacia
17 lista = []
18
19 # Verifica una lista con valores aleatorios
20 for indice in range (1,11):
21     lista.append(random.randint(1,10))
22
23 # Ordenar la lista
24 lista.sort()
25
26 # Recorrer el vector ordenado
27 for numero in lista:
28     print(numero,end="")
```

Resultado:

```
PS D:\Python_EBGA1\Python_EB>Listas_SLTM> & C:/Users/HP/AppData/Local/Programs/Python/Python314/python.exe d:/Python_EBGA1/Python_EB/Listas_SLTM/E5_SLTM.py
11577778910
PS D:\Python_EBGA1\Python_EB>Listas_SLTM> █
```

Programa 6: Crea un programa que pida un número al usuario un número de mes (por ejemplo, el 4) y diga cuántos días tiene (por ejemplo, 30) y el nombre del mes. Debes usar listas. Para simplificarlo vamos a suponer que febrero tiene 28 días. Observación: Al ejecutar su programa ingresar el número de mes de su cumpleaños y verificar los días del año 2026.

```
E6_SLTM.py > ...
1 # Ejercicios de Listas
2 # E6_SLTM
3 # SLTM
4
5 import pyfiglet
6 from colorama import init, Fore, Back, Style
7
8 init()
9
10 titulo = pyfiglet.figlet_format("Sara")
11 print(Fore.CYAN + titulo + Style.BRIGHT)
12
13 print(Fore.MAGENTA + "Texto en color magenta")
14
15 # Crear lista con los dias ultimos de cada mes
16 dias = [31,28,31,30,31,30,31,31,30,31,30,31]
17 # Crear lista con los meses del año
18 nombre_mes = ['Enero','Febrero','Marzo','Abril','Mayo','Junio','Julio','Agosto','Septiembre','Octubre','Noviembre','Diciembre']
19
20 # Verificacion del mes de acuerdo al numero y nombre
21 while True:
22     mes = int(input("Introduce un mes (1-12): "))
23     if mes < 1 or mes >12:
24         print("Error mes incorrecto.")
25     if mes>=1 and mes<= 12: break
26
27 print("El mes de ", nombre_mes[mes-1],"tiene ",dias[mes-1],"dias!")
```

Resultado:

```
PS D:\Python_EBGA1\Python_EB>Listas_SLTM> & C:/Users/HP/AppData/Local/Programs/Python/Python314/python.exe d:/Python_EBGA1/Python_EB/Listas_SLTM/E6_SLTM.py
Introduce un mes (1-12): 8
El mes de Agosto tiene 31 dias!
PS D:\Python_EBGA1\Python_EB>Listas_SLTM> |
```

Programa 7: Programa que declare tres listas 'lista1', 'lista2' y 'lista3' de cinco enteros cada uno, pida valores para 'lista1' y 'lista2' y calcule lista3=lista1+lista2. Observación: Al ejecutar su programa ingresar su clave para Introducir el elemento 1 del vector1

```
E7_SLTM.py > ...
1  # Ejercicios de Listas
2  # E7_SLTM
3  # SLTM
4
5  import pyfiglet
6  from colorama import init, Fore, Back, Style
7
8  init()
9
10 titulo = pyfiglet.figlet_format("Sara")
11 print(Fore.CYAN + titulo + Style.BRIGHT)
12
13 print(Fore.MAGENTA + "Texto en color magenta")
14 # Crear listas vacías donde ingresarán 5 enteros
15 lista1 = []
16 lista2 = []
17 lista3 = []
18
19 # Recorrer listas para ingresar elementos
20 for indice in range(1,6):
21     lista1.append(int(input("Introduce el elemento %d del vector1:"
22
23 for indice in range(1,6):
24     lista2.append(int(input("Introduce el elemento %d del vector2:"
25
26     for indice in range(0,5):
27         lista3.append(lista1[indice] + lista2[indice])
28
29 print("La suma de los vectores es: ")
30 for numero in lista3:
31     print(numero, " ", end="")
```

Resultado:

```
PS D:\Python_EBGA1\Python_EB>Listas_SLTM> & C:/Users/HP/AppData/Local/Programs/Python/Python314/python.exe d:/Python_EBGA1/Python_EB/Listas_SLTM/E7_SLTM.py
Introduce el elemento 1 del vector1: 25
Introduce el elemento 2 del vector1: 25
Introduce el elemento 3 del vector1: 25
Introduce el elemento 4 del vector1: 25
Introduce el elemento 5 del vector1: 25
Introduce el elemento 1 del vector2: 45
Introduce el elemento 2 del vector2: 45
Introduce el elemento 3 del vector2: 45
Introduce el elemento 4 del vector2: 45
Introduce el elemento 5 del vector2: 45
La suma de los vectores es:
70 70 70 70 70
```


Programa 8: # Queremos guardar los nombres y edades de los alumnos de un curso. Realiza un programa que introduzca el nombre y la edad de cada alumno. El proceso de lectura de datos terminará cuando se introduzca como nombre un asterisco (*) Al finalizar se mostrarán los siguientes datos:

- * Todos los alumnos mayores de edad.
- * Los alumnos mayores (los que tienen más edad)

Observación: Al ejecutar su programa debe de ingresar su nombre y su edad, ingresar 5 nombres de sus compañeros.

```
E8_SLTM.py > ...
1  # Ejercicios de Listas
2  # E8_SLTM
3  # SLTM
4
5  import pyfiglet
6  from colorama import init, Fore, Back, Style
7
8  init()
9
10 titulo = pyfiglet.figlet_format("Sara")
11 print(Fore.CYAN + titulo + Style.BRIGHT)
12
13 print(Fore.MAGENTA + "Texto en color magenta")
14
15 # Crear listas vacias para su utilidad
16 nombres = []
17 edades = []
18
19 # Inicializar las listas hasta que se introduzca "*"
20 while True:
21     nombre = input("Indique el nombre de un alumno: ")
22     if nombre != "*":
23         # conteo de cuantos alumnos se han ingresado
24         nombres.append(nombre)
25         # Ingreso de conteo de las edades
26         edades.append(int(input("Indique su edad: ")))
27     if nombre == "*": break;
28
29 # Calcular la edad maxima
30 edad_max = max(edades)
31 # Alumnos mayores de edad
32 print("Alumnos mayores de edad")
33 print("=====")
34 for nombre, edad in zip(nombres, edades):
35     if edad >= 18:
36         print(nombre)
37
38 # Mostrar los alumnos mayores de edad
39 print("Alumnos mayores")
40 print("=====")
41 for nombre, edad in zip(nombres, edades):
42     if edad == edad_max:
43         print(nombre)
```

Resultado:

```
PS D:\Python_EBGA1\Python_EB>Listas_SLTM> & C:/Users/HP/AppData/Local/Programs/Python/Python314/python.exe d:/Python_EBGA1/Python_EB/Listas_SLTM/E8_SLTM.py
Indique el nombre de un alumno: Sara
Indique su edad: 18
Indique el nombre de un alumno: Leslie
Indique su edad: 17
Indique el nombre de un alumno: Karen
Indique su edad: 17
Indique el nombre de un alumno: Carlitos
Indique su edad: 17
Indique el nombre de un alumno: Lucia
Indique su edad: 17
Indique el nombre de un alumno: *
Alumnos mayores de edad
=====
Sara
Alumnos mayores
=====
Sara
PS D:\Python_EBGA1\Python_EB>Listas_SLTM>
```

Programa 9: Queremos guardar la temperatura mínima y máxima de 5 días. Realiza un programa que de la siguiente información:

- * La temperatura media de cada día
- * Los días con menos temperatura
- * Se lee una temperatura por teclado y se muestran los días cuya temperatura máxima coincide con ella.
- * Si no existe ningún día se muestra un mensaje de información.

Recorrido para rellenar la tabla (5 días con temp máxima y mínima)

```
# Ejercicios de Listas
# E9_SLTM
# SLTM

import pyfiglet
from colorama import init, Fore, Back, Style

init()

titulo = pyfiglet.figlet_format("Sara")
print(Fore.CYAN + titulo + Style.BRIGHT)

print(Fore.MAGENTA + "Texto en color magenta")

# Crear lista vacia donde luego se guardará la cantidad de temperatura
temperaturas= []

# Ciclo para conocer de pregunta entre temperatura min y max
for indice in range(1,6):
    # Almacenar las temperaturas
    temperatura = []

    #Solicitud de temperaturas
    temperatura.append(int(input("Día %d. Temperatura maxima: " % indice)))
    temperatura.append(int(input("Día %d. Temperatura minimo: " % indice)))
    temperaturas.append(temperatura)

# Mostrar temperatura media
print("Temperaturas medias")
print("=====")

# Indice que indica la temperatura media
indice = 1
# Ciclo para la temperatura media
for temperatura in temperaturas:
    print("Día %d, Temperatura media: %f: " %(indice,sum(temperatura)/len(temperatura)))
    indice += 1

# Calcular temperatura minima mas pequeña
# Tomando en cuenta que es la primera ingresada
temp_min = temperaturas[0][1];
# Inicializacion para comparar
for temperatura in temperaturas:
    if temperatura[1] < temp_min:
        temp_min = temperatura[1]

# Mostrar los días con menos temperatura
print("Días con menos temperaturas")
print("=====")
```

```

50
51 # Solicitar la temperatura
52 temp_max = int(input("Ingrese una temperatura: "))
53 #Indice para indicar temperaturas
54 indice = 1
55 for temperatura in temperaturas:
56     # Coincidir cual es la mayor temperatura
57     if temperatura[0] == temp_max:
58         print("Dia %d" % indice)
59         existe_temperatura = True
60         indice +=1
61 # si no hay coincidencia, enseñar el siguiente mensaje
62 if not existe_temperatura:
63     print("No hay ningun dia con dicha temperatura")

```

Resultado:

```

PS D:\Python_EBGA1\Python_EB>Listas_SLTM> & C:/Users/HP/AppData/Local/Programs/Python/Python314/python.exe d:/Python_EBGA1/Python_EB/Listas_SLTM/E9_SLTM.py
Dia 1. Temperatura maxima: 35
Dia 1. Temperatura minimo: 20
Dia 2. Temperatura maxima: 39
Dia 2. Temperatura minimo: 26
Dia 3. Temperatura maxima: 45
Dia 3. Temperatura minimo: 32
Dia 4. Temperatura maxima: 36
Dia 4. Temperatura minimo: 20
Dia 5. Temperatura maxima: 45
Dia 5. Temperatura minimo: 32
Temperaturas medias
=====
Dia 1, Temperatura media: 38.500000:
Dia 2, Temperatura media: 38.500000:
Dia 3, Temperatura media: 38.500000:
Dia 4, Temperatura media: 38.500000:
Dia 5, Temperatura media: 38.500000:
Dias con menos temperaturas
=====

```

Repositorio 1:

https://github.com/saratipaz-cpu/GA1-Listas_SLTM_3U/blob/main/GA1_Listas.pdf

Parte 2:

Programa 1:

```
# Fundamentos de POO en Python

# Ejercicios de Listas
# E1_SLTM
# SLTM

import pyfiglet
from colorama import init, Fore, Back, Style

init()

titulo = pyfiglet.figlet_format("Sara")
print(Fore.CYAN + titulo + Style.BRIGHT)

print(Fore.MAGENTA + "Texto en color magenta")

# Se crea una clase llamada "Carro"
class Carro:
    # Se crea un constructor de la clase
    # Se ejecuta automaticamente al crear un objeto
    def __init__(self, marca, modelo):

        # Se crea el atributo 'marca'
        # Y se guarda el valor recibido en el parametro 'marca'
        self.marca = marca

        # Se crea el atributo 'modelo' y se guarda el valor
        # recibido en el parametro 'modelo'
        self.modelo = modelo

# Crear el objeto 'carrito'
# La marca sera 'Toyota' y el modelo es 'Corolla'
carrito = Carro(marca="Toyota", modelo= "Corolla")

# Se imprime el atributo 'marca' del objeto 'carrito'
# Si se desea que se muestre la marca o modelo debe de
# agregarse .atributo
print(carrito.marca) # Resultado: Toyota
```

Resultado:

```
PS C:\Users\Computacion\Documents\Fundamentos_P00> & C:/Users/Computacion/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Computacion/Documents/Fundamentos_P00/Poo1.py
Toyota
PS C:\Users\Computacion\Documents\Fundamentos_P00> █
```

Programa 2:

```
1  # Fundamentos de P00 en Python
2  # Clase vacia
3
4  # Ejercicios de Listas
5  # E2_SLTM
6  # SLTM
7
8  import pyfiglet
9  from colorama import init, Fore, Back, Style
10
11  init()
12
13  titulo = pyfiglet.figlet_format("Sara")
14  print(Fore.CYAN + titulo + Style.BRIGHT)
15
16  print(Fore.MAGENTA + "Texto en color magenta")
17  # Se crea una clase llamada 'Perro'
18  class Perro:
19
20      # pass: Indica que la clase esta vacia y que por el momento
21      # No tiene atributos ni metodos definidos
22      pass
```

Resultado:

```
hon/Python314/python.exe c:/Users/Computacion/Documents/Fundamentos_P00/Poo2.py
PS C:\Users\Computacion\Documents\Fundamentos_P00> 
```

Ejercicio 3:

```
Poo3.py > ...
1  # Fundamentos de POO en Python
2
3  # E1_SLTM
4  # SLTM
5
6  import pyfiglet
7  from colorama import init, Fore, Back, Style
8
9  init()
10
11 titulo = pyfiglet.figlet_format("Sara")
12 print(Fore.CYAN + titulo + Style.BRIGHT)
13
14 print(Fore.MAGENTA + "Texto en color magenta")
15
16 # Atributos
17
18 # Se crea una clase llamada Perro
19 # la clase es una plantilla para crear objetos
20 # En esta clase se crean objetos de tipo 'Perro'
21 class Perro:
22
23     # Se define el constructor de la clase
24     # Este metodo se ejecuta automaticamente cuando se crea un objeto
25     # Self: representa el objeto actual
26     # Nombre: recibe el nombre del perro
27     # Raza: recibe la raza del perro
28     def __init__(self, nombre, raza):
29
30         # Se crea el atributo nombre dentro del objeto
31         # Self.nombre pertenece al objeto
32         # Se almacena el valor recibido en el parametro 'nombre'
33         self.nombre = nombre
34
35         # Se crea el atributo raza dentro del objeto
36         # Self.raza pertenece al objeto
37         # Se almacena el valor recibido en el parametro 'raza'
38         self.raza = raza
```


Resultado:

```
PS C:\Users\Computacion\Documents\Fundamentos_P00> & C:/Users/Computacion/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Computacion/Documents/Fundamentos_P00/Poo3.py
PS C:\Users\Computacion\Documents\Fundamentos_P00> █
```

Programa 4:

```
Poo4.py > ...
1  # Fundamentos de POO en Python
2
3  # E4_SLTM
4  # SLTM
5
6  import pyfiglet
7  from colorama import init, Fore, Back, Style
8
9  init()
10
11 titulo = pyfiglet.figlet_format("Sara")
12 print(Fore.CYAN + titulo + Style.BRIGHT)
13
14 print(Fore.MAGENTA + "Texto en color magenta")
15
16 # Métodos dentro de una clase
17
18 # Se crea una clase llamada 'Perro'
19 class Perro:
20
21     # Un metodo es una funcion que esta adentro de una clase
22     # En este caso, se crea un metodo llamado 'ladrar'
23     # self: representa al objeto que ejecutara el metodo
24     def ladrar(self):
25
26         # Esta instruccion muestra el mensaje
27         # En este caso, representa el sonido que hace el perro
28         print("Guau Guau")
29
```

Resultado:

```
PS C:\Users\Computacion\Documents\Fundamentos_P00> & C:/Users/Computacion/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Computacion/Documents/Fundamentos_P00/Poo3.py
PS C:\Users\Computacion\Documents\Fundamentos_P00> █
```

Programa 5:

```
1  # Fundamentos de POO en Python
2
3  # E5_SLTM
4  # SLTM
5
6  import pyfiglet
7  from colorama import init, Fore, Back, Style
8
9  init()
10
11 titulo = pyfiglet.figlet_format("Sara")
12 print(Fore.CYAN + titulo + Style.BRIGHT)
13
14 print(Fore.MAGENTA + "Texto en color magenta")
15
16 # Se crea una clase
17 # NombreClase debe de reemplazarse por el nombre que deseamos dar a la clase
18 class NombreClase:
19
20     # Se define el metodo constructor
21     # __init__ se ejecuta automaticamente al crear un objeto
22     # self: representa al objeto actual
23     # atributo1 y atributo2 son parametros que recibira el contructor
24     def __init__(self, atributo1, atributo2):
25
26         # Se crea el atributo 'atributo1' dentro del objeto
27         # Se almacena el valor recibido en el parametro 'atributo1'
28         self.atributo1 = atributo1
29
30         # Se crea el atributo 'atributo2' dentro del objeto
31         # Se almacena el valor recibido en el parametro 'atributo2'
32         self.atributo2 = atributo2
33
```

Resultado:

```
PS C:\Users\Computacion\Documents\Fundamentos_P00> & C:/Users/Computacion/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Computacion/Documents/Fundamentos_P00/Poo4.py
PS C:\Users\Computacion\Documents\Fundamentos_P00> █
```

Programa 6:

```
1  # Fundamentos de POO en Python
2
3  # Ejemplo SLTM
4  # SLTM
5
6  import pyfiglet
7  from colorama import init, Fore, Back, Style
8
9  init()
10
11 titulo = pyfiglet.figlet_format("Sara")
12 print(Fore.CYAN + titulo + Style.BRIGHT)
13
14 print(Fore.MAGENTA + "Texto en color magenta")
15
16 # Se crea una clase llamada "Persona"
17 # La clase sirve como plantilla para crear objetos de un tipo 'Persona'
18 class Persona:
19     # Se define el metodo constructor
20     # __init__ se ejecuta automaticamente al crear un objeto
21     # self: representa al objeto actual
22     # nombre y edad son parametros que recibira el constructor
23     def __init__(self, nombre, edad):
24         # Se crea el atributo 'nombre' dentro del objeto
25         # Se almacena el valor recibido en el parametro 'nombre'
26         self.nombre = nombre
27
28         # Se crea el atributo 'edad' dentro del objeto
29         # Se almacena el valor recibido en el parametro 'edad'
30         self.edad = edad
31
32 # Se crea un objeto llamado 'persona1' utilizando la clase Persona
33 # Se envian dos datos al constructor:
34 # nombre = "Carlitos"
35 # edad = 30
36 persona1 = Persona(nombre='Carlitos', edad=30)
37
38 # Se imprime el valor almacenado en el atributo 'nombre' del objeto persona1
39 # el resultado que aparecera en pantalla sera: Carlitos
40 print(persona1.nombre)
```

Resultado:

```
PS C:\Users\Computacion\Documents\Fundamentos_P00> & C:/Users/Computacion/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Computacion/Documents/Fundamentos_P00/Poo6.py
Carlitos
PS C:\Users\Computacion\Documents\Fundamentos_P00> █
```

Programa 7:

```
1  # Fundamentos de P00 en Python
2
3  # E7_SLTM
4  # SLTM
5
6  import pyfiglet
7  from colorama import init, Fore, Back, Style
8
9  init()
10
11 titulo = pyfiglet.figlet_format("Sara")
12 print(Fore.CYAN + titulo + Style.BRIGHT)
13
14 print(Fore.MAGENTA + "Texto en color magenta")
15
16 # Atributos de instancia
17
18 # Se crea una clase llamada 'Carro'
19 class Carro:
20
21     # Se define el constructor de la clase
22     # Este metodo se ejecuta automaticamente cuando se crea un objeto
23     # Self: representa al objeto actual
24     # color: es un objeto que recibira el objeto
25     def __init__(self, color):
26
27         # Se crea un atributo de instancia llamado 'color'
28         # El atributo pertenece unicamente al objeto que se esta creando
29         # Se almacena el valor recibido en el parametro 'color'
30         self.color = color
```

Resultado:

```
Carritos
PS C:\Users\Computacion\Documents\Fundamentos_P00> & C:/Users/Computacion/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Computacion/Documents/Fundamentos_P00/Poo7.py
PS C:\Users\Computacion\Documents\Fundamentos_P00> |
```


Programa 8:

```
1  # Fundamentos de POO en Python
2
3  # E8_SLTM
4  # SLTM
5
6  import pyfiglet
7  from colorama import init, Fore, Back, Style
8
9  init()
10
11 titulo = pyfiglet.figlet_format("Sara")
12 print(Fore.CYAN + titulo + Style.BRIGHT)
13
14 print(Fore.MAGENTA + "Texto en color magenta")
15
16 # Atributos de instancia
17
18 # Se crea una clase llamada 'Carro'
19 class Carro:
20
21     # Se crea un atributo de clase llamado 'rueda'
22     # Este atributo pertenece a la clase Carro
23     # Todos los objetos creados a partir de esta clase comparten
24     # este mismo valor :D
25     ruedas = 4
26
27 # Se muestra el valor del atributo de la clase 'Carro'
28 # No es necesario crear un objeto para acceder a él
29 # Se accede directamente utilizando el nombre de la clase.atributo
30 print(Carro.ruedas)
```

Resultado:

```
PS C:\Users\Computacion\Documents\Fundamentos_P00> & C:/Users/Computacion/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Computacion/Documents/Fundamentos_P00/Poo8.py
4
PS C:\Users\Computacion\Documents\Fundamentos_P00> 
```

Programa 9:

```
P009.py > ...
1  # Fundamentos de P00 en Python
2
3  # E9 SLTM
4  # SLTM
5
6  import pyfiglet
7  from colorama import init, Fore, Back, Style
8
9  init()
10
11 titulo = pyfiglet.figlet_format("Sara")
12 print(Fore.CYAN + titulo + Style.BRIGHT)
13
14 print(Fore.MAGENTA + "Texto en color magenta")
15
16 # Metodos de instancia
17
18 # Se crea una clase llamada 'Perro'
19 class Perro:
20
21     # Un metodo es una funcion que esta adentro de una clase
22     # En este caso, se crea un metodo llamado 'ladrar'
23     # self: representa al objeto que ejecutara el metodo
24     def ladrar(self):
25
26         # Esta instruccion muestra el mensaje
27         # En este caso, representa el sonido que hace el perro
28         print("Guau Guau")
29
30 # Se crea un objeto llamado 'mi_perro' a partir de la clase 'Perro'
31 mi_perro = Perro()
32
33 # El objeto 'mi_perro' ejecuta el método 'ladrar()'
34 # Al ejecutar, se mostrara el mensaje 'Guau Guau'
35 mi_perro.ladrar()
```

Resultado:

```
PS C:\Users\Computacion\Documents\Fundamentos_P00> & C:/Users/Computacion/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Computacion/Documents/Fundamentos_P00/Poo9.py
Guau Guau
PS C:\Users\Computacion\Documents\Fundamentos_P00> █
```

Programa 10:

```
Poo10.py > ...
1  # Fundamentos de POO en Python
2
3  # E10_SLTM
4  # SLTM
5
6  import pyfiglet
7  from colorama import init, Fore, Back, Style
8
9  init()
10
11 titulo = pyfiglet.figlet_format("Sara")
12 print(Fore.CYAN + titulo + Style.BRIGHT)
13
14 print(Fore.MAGENTA + "Texto en color magenta")
15
16 # Metodo con parametros
17
18 # Se crea la clase llamada 'Persona'
19 class Persona:
20
21     # Se define un metodo llamado 'saludar()'
22     # self: representa al objeto que ejecuta el metodo
23     # 'nombre': es un parametro que recibira informacion
24     # cuando el metodo sea llamado
25     def saludar(self, nombre):
26
27         # Se muestra el mensaje 'Salida de datos'
28         # La f antes de las comillas: indica un f-string
29         # Permite insertar el valor de la variable 'nombre'
30         # dentro del texto
31         print(f"Hola {nombre} !" )
32
33 # Se crea un objeto llamado 'p' a partir de la clase 'Persona'
34 p = Persona()
35
36 # El objeto 'p' ejecuta el metodo 'saludar()'
37 # Se envia el valor 'Maulwurf' como argumento al parametro 'nombre'
38 p.saludar("Maulwurf")
```

Resultado:

```
PS C:\Users\Computacion\Documents\Fundamentos_P00> & C:/Users/Computacion/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Computacion/Documents/Fundamentos_P00/Poo10.py
Hola Maulwurf !
PS C:\Users\Computacion\Documents\Fundamentos_P00> █
```

Programa 11:

```
Poo11.py > CuentaBancaria
1  # E11_SLTM
2  # SLTM
3
4  import pyfiglet
5  from colorama import init, Fore, Back, Style
6
7  init()
8
9  titulo = pyfiglet.figlet_format("Sara")
10 print(Fore.CYAN + titulo + Style.BRIGHT)
11
12 print(Fore.MAGENTA + "Texto en color magenta")
13
14 # Fundamentos de POO en Python
15
16 # Se crea una clase llamada 'CuentaBancaria'
17 # Esta clase servirá como plantilla para crear objetos que representen
18 # cuentas bancarias
19 class CuentaBancaria:
20
21     # Constructor de la clase
22     ... # Se ejecuta automáticamente cuando se crea un objeto
23     ... # 'saldo' es un dato que recibirá el objeto
24     def __init__(self, saldo):
25
26         # Se crea un atributo privado llamado '__saldo'
27         # Los dos guiones bajos indican encapsulamiento
28         # El saldo queda protegido dentro del objeto
29         self.__saldo = saldo
30
31     # Se crea un método llamado 'mostrar_saldo'
32     # Este método permitirá visualizar el saldo almacenado
33     def mostrar_saldo(self):
34
35         # Se muestra el valor del atributo privado __saldo
36         print(self.__saldo)
```

Resultado:

```
PS C:\Users\Computacion\Documents\Fundamentos_P00> & C:/Users/Computacion/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Computacion/Documents/Fundamentos_P00/Poo11.py
PS C:\Users\Computacion\Documents\Fundamentos_P00> █
```

Programa 12:

```
Poo12.py > ...
1  # E12_SLTM
2  # SLTM
3
4  import pyfiglet
5  from colorama import init, Fore, Back, Style
6
7  init()
8
9  titulo = pyfiglet.figlet_format("Sara")
10 print(Fore.CYAN + titulo + Style.BRIGHT)
11
12 print(Fore.MAGENTA + "Texto en color magenta")
13
14 # Se crea una clase llamada 'Animal'
15 # esta sera la clase padre o superclase
16 class Animal:
17     # Se define un metodo llamado 'sonido'
18     # Este metodo devuelve un solo sonido generico
19     def sonido(self):
20         # Retorna el texto 'sonido generico'
21         return "Sonido generico"
22
23 # Se crea una clase llamada 'Perro'
24 # Entre paréntesis se indica que Perro hereda de Animal
25 # Perro se convierte en la clase hija o subclase
26 class Perro(Animal):
27     # Se crea nuevamente el método sonido
28     # este metodo reemplaza al metodo sonido heredado de Animal
29     def sonido(self):
30         return "Guau"
31
32 # Se crea un objeto llamado 'mi_perro' utilizando la clase Perro
33 mi_perro = Perro()
34
35 # Se ejecuta el metodo sonido() del objeto
36 # Como Perro tiene su propio metodo sonido() se ejecuta el de la clase
37 # Perro y no el de Animal
38 print(mi_perro.sonido())
39
40
41
42
```


Resultado:

```
PS C:\Users\Computacion\Documents\Fundamentos_P00> & C:/Users/Computacion/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Computacion/Documents/Fundamentos_P00/Poo12.py
Guau
PS C:\Users\Computacion\Documents\Fundamentos_P00> █
```

Programa 13:

```
Poo13.py > Perro
1  # E13_SLTM
2  # SLTM
3
4  import pyfiglet
5  from colorama import init, Fore, Back, Style
6
7  init()
8
9  titulo = pyfiglet.figlet_format("Sara")
10 print(Fore.CYAN + titulo + Style.BRIGHT)
11
12 print(Fore.MAGENTA + "Texto en color magenta")
13
14 # Fundamentos de POO en Python
15
16 # Se crea una clase llamada Perro
17 class Perro:
18
19     # Metodo sonido
20     # Devuelve el sonido caracteristico del perro
21     def sonido(self):
22
23         # Retorna la palabra Guau
24         return "Guau"
25
26 # Se crea una clase llamada Gato
27 class Gato:
28
29     # Metodo sonido
30     # Tiene el mismo nombre que el metodo de la clase Perro
31     def sonido(self):
32
33         # Retorna la palabra "miau"
34         return "Miau"
35
36 # Se crea una lista llamada "Animales"
37 # Dentro de la lista se almacena objetos de diferentes clases
38 animales = [Perro(), Gato()]
39
40 # Se recorre cada objeto almacenado en la lista
41 for animal in animales:
42
43     # Se ejecuta el metodo sonido() de cada objeto
44     # Dependiendo del objeto, el resultado sera diferente
45     print(animal.sonido())
```

Resultado:

```
Guau
PS C:\Users\Computacion\Documents\Fundamentos_P00> & C:/Users/Computacion/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Computacion/Documents/Fundamentos_P00/Poo13.py
Guau
Miau
PS C:\Users\Computacion\Documents\Fundamentos_P00> █
```

Programa 14:

```
Poo14.py > ...
1  # E14_SLTM
2  # SLTM
3
4  import pyfiglet
5  from colorama import init, Fore, Back, Style
6
7  init()
8
9  titulo = pyfiglet.figlet_format("Sara")
10 print(Fore.CYAN + titulo + Style.BRIGHT)
11 print(Fore.MAGENTA + "Texto en color magenta")
12
13 # Fundamentos de POO en Python
14
15 # Se crea una clase llamada CuentaBancaria
16 class CuentaBancaria:
17
18     # Constructor de la clase
19     def __init__(self, saldo):
20
21         # Atributo privado
22         # El usuario no debe modificarlo directamente
23         self.__saldo = saldo
24
25     # Metodo publico para depositar dinero
26     def depositar(self, cantidad):
27
28         # proceso interno oculto al usuario
29         self.__saldo += cantidad
30
31         # Imprimir mensaje de deposito realizado
32         print("Deposito realizado correctamente")
33
34     # Metodo publico para consultar saldo
35     def consultar_saldo(self):
36
37         # devuelve el saldo actual
38         return self.__saldo
39
40 # Crear objeto
41 cuenta = CuentaBancaria(1000)
42
43 # Depositar dinero
44 cuenta.depositar(500)
45
46 # Consultar saldo
47 print("Saldo actual ", cuenta.consultar_saldo())
```

Resultado:

```
PS C:\Users\Computacion\Documents\Fundamentos_P00> & C:/Users/Computacion/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Computacion/Documents/Fundamentos_P00/Poo14.py
Deposito realizado correctamente
Saldo actual 1500
PS C:\Users\Computacion\Documents\Fundamentos_P00> █
```

Programa 15:

```
Poo15.py > ...
1  # E15_SLTM
2  # SLTM
3
4  import pyfiglet
5  from colorama import init, Fore, Back, Style
6
7  init()
8
9  titulo = pyfiglet.figlet_format("Sara")
10 print(Fore.CYAN + titulo + Style.BRIGHT)
11 print(Fore.MAGENTA + "Texto en color magenta")
12 # Fundamentos de POO en Python
13 # Clase Alumno
14 class Alumno:
15
16     # Constructor con los atributos nombre, nota1 y nota 2
17     def __init__(self, nombre, nota1, nota2):
18
19         # Constructor de la clase
20         # Se ejecuta automáticamente cuando se crea un objeto
21         # 'nombre' es un dato que recibirá el objeto
22         self.nombre = nombre
23
24         # Constructor de la clase
25         # Se ejecuta automáticamente cuando se crea un objeto
26         # 'nota1' es un dato que recibirá el objeto
27         self.nota1 = nota1
28
29         # Constructor de la clase
30         # Se ejecuta automáticamente cuando se crea un objeto
31         # 'nota2' es un dato que recibirá el objeto
32         self.nota2 = nota2
33
34     #Metodo para calcular promedio
35     def mostrar_promedio(self):
36
37         # Se toman las dos notas y las dividen por la cantidad dada
38         promedio = (self.nota1 + self.nota2) / 2
39
40         # Retorna el resultado
41         return promedio
42
43     # Crear el objeto alumno1
44     alumno1 = Alumno("Sara", 80,90)
45
46     # Se muestra el promedio del alumno1
47     print("Promedio: ", alumno1.mostrar_promedio())
```

Resultado:

```
PS C:\Users\Computacion\Documents\Fundamentos_P00> & C:/Users/Computacion/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Computacion/Documents/Fundamentos_P00/Poo15.py
Promedio: 85.0
PS C:\Users\Computacion\Documents\Fundamentos_P00> █
```

Programa 16:

Poo16.py > ...

```
1  # E16_SLTM
2  # SLTM
3
4  import pyfiglet
5  from colorama import init, Fore, Back, Style
6
7  init()
8
9  titulo = pyfiglet.figlet_format("Sara")
10 print(Fore.CYAN + titulo + Style.BRIGHT)
11
12 print(Fore.MAGENTA + "Texto en color magenta")
13
14 # Fundamentos de POO en Python
15
16 # Se crea clase llaada CuentaBancaria
17 # Esta clase servira como plantilla para crear cuentas bancarias
18 class CuentaBancaria:
19
20     # Constructor de la clase
21     # titular: recibe el nombre del propietario de la cuenta
22     # saldo= 0 indica que si no se envia un saldo, automaticamen
23     def __init__(self, titular, saldo=0):
24
25         # Atributo publico que almacena el nombre del titular
26         self.titular = titular
27
28         # Atributo privado que almacena el saldo
29         # los dos guiones bajos indican encapsulamiento
30         self.__saldo = saldo
31
32     # Metodo para depositar dinero
33     def depositar(self, cantidad):
34
35         # Suma la cantidad recibida al saldo actual
36         self.__saldo += cantidad
37
38         # Muestra un mensaje confirmando el deposito
39         print(f"Deposito de {cantidad} realizado")
40
41     # Metodo para retirar dinero
42     def retirar(self, cantidad):
43
44         # Verifica si existe saldo suficiente
45         if cantidad <= self.__saldo:
46
47             # Resta la cantidad de saldo suficiente
48             self.__saldo -= cantidad
```

```
48         self.__saldo -= cantidad
49
50         # Muestra mensaje de retiro exitoso
51         print(f"Retiro de {cantidad} realizado")
52
53     else:
54
55         # Muestra mensaje si el saldo es insuficiente
56         print("Saldo insuficiente")
57
58     # Metodo para mostrar el saldo actual
59     def mostrar_saldo(self):
60
61         # Muestra el saldo disponible
62         print(f"Saldo disponible: {self.__saldo}")
63
64     # Se crea un objeto llamado cuenta
65     # Titular: Sara
66     # Saldo inicial : 100
67     cuenta = CuentaBancaria("Sara", 100)
68
69     # Se deposita 50 al saldo
70     cuenta.depositar(50)
71
72     # Se retiran 30 del saldo
73     cuenta.retirar(30)
74
75     # Se muestra el saldo final
76     cuenta.mostrar_saldo()
```

Resultado:

```
PS C:\Users\Computacion\Documents\Fundamentos_POO> & C:/Users/Computacion/AppData/Local/Programs/Python/Python314/python.exe c:/Users/Computacion/Documents/Fundamentos_POO/Poo16.py
Deposito de 50 realizado
Retiro de 30 realizado
Saldo disponible: 120
PS C:\Users\Computacion\Documents\Fundamentos_POO> █
```

Repositorio 2:

https://github.com/saratipaz-cpu/PC2_SLTM_3U_POO-Parte2